



TECNOLÓGICO
NACIONAL DE MÉXICO



Tecnológico Nacional de México
Instituto Tecnológico Superior de Misantla
Ingeniería en Sistemas Computacionales

Materia
Taller de Bases de Datos

Unidad 2
Lenguaje de Manipulación de Datos (DML)

Actividad
CRUD

Presenta
Mancera Herrera Luis Santiago

Titular:
Dr. Irahan Otoniel José Guzmán

Misantla, Veracruz

16 – 09 – 2026

Índice

Índice de figuras.....	3
Introducción	4
Restaurante.....	5
CREATE TABLE comidas	5
INSERT INTO Refresco y Comida	6
UPDATE comidas	7
DELETE comidas	7
Escuela	8
CREATE TABLE alumnos.....	8
INSERT alumnos.....	9
UPDATE alumnos.....	10
DELETE Carlos Ruiz.....	10
Tienda Escolar.....	11
CREATE TABLE productos	11
INSERT INTO productos	12
UPDATE productos.....	13
DELETE Galletas.....	13
Conclusión.....	14

Índice de figuras

Figura 1: Preparación de la Base de Datos.....	5
Figura 2: CREATE TABLE comidas	5
Figura 3: INSERT INTO Refresco y Comida	6
Figura 4: UPDATE precio	7
Figura 5: DELETE de Refresco	7
Figura 6: CREATE TABLE alumnos	8
Figura 7: INSERT INTO alumnos.....	9
Figura 8: UPDATE promedio.....	10
Figura 9: DELETE Carlos Ruiz	10
Figura 10: CREATE TABLE productos	11
Figura 11: INSERT INTO productos.....	12
Figura 12: UPDATE existencia	13
Figura 13: DELETE Galletas	13

Introducción

En la siguiente práctica se utilizó la plataforma **Google Colab** como entorno de trabajo para desarrollar y ejecutar código en Python orientado al manejo de bases de datos. Mediante las bibliotecas **SQLite3** y **Pandas**, se realizaron diferentes actividades que permitieron crear, administrar y consultar bases de datos de manera sencilla.

Durante la práctica se trabajó con distintos ejemplos relacionados con un restaurante, una escuela y una tienda de productos, creando tablas para almacenar información importante y aplicando operaciones básicas del lenguaje SQL. Entre las acciones realizadas se encuentran la creación de tablas (**CREATE**), la inserción de registros (**INSERT**), la actualización de datos (**UPDATE**) y la eliminación de información (**DELETE**).

El propósito de esta actividad fue poner en práctica el uso de bases de datos mediante situaciones reales y sencillas, aprendiendo a registrar, organizar y gestionar información de forma estructurada. Asimismo, se buscó familiarizarse con el entorno de **Google Colab**.

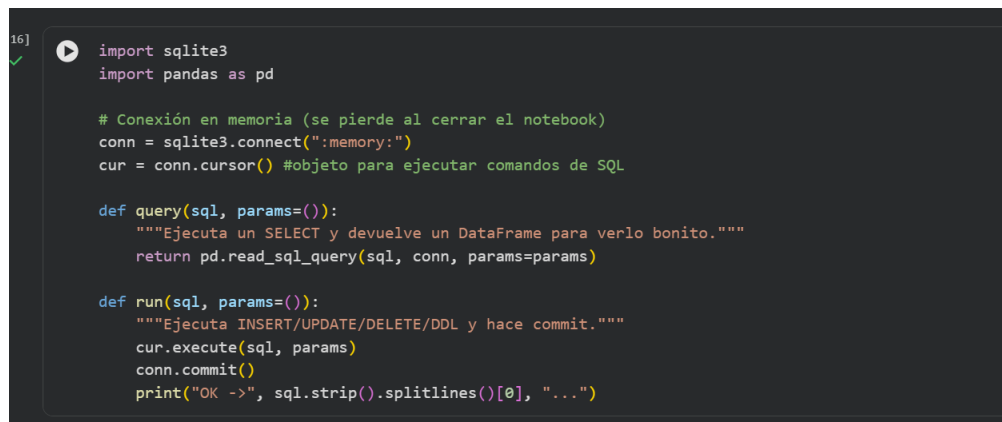
Restaurante

Link de la plataforma Colab: [Restaurante.ipynb - Colab](#)

Preparación de la base de datos

Para todas estas prácticas utilizamos la misma preparación para la base de datos, en la que primero importamos las bibliotecas **sqlite3** y **pandas**, necesarias para la conexión a bases de datos. Posteriormente, se crea una base de datos temporal en memoria mediante `sqlite3.connect(":memory:")` y se define un cursor para ejecutar instrucciones SQL.

Así mismo se definen las funciones para la visualización de las tablas y poder ejecutar las operaciones de modificación de datos como INSERT, UPDATE y DELETE



```
16] import sqlite3
import pandas as pd

# Conexión en memoria (se pierde al cerrar el notebook)
conn = sqlite3.connect(":memory:")
cur = conn.cursor() #objeto para ejecutar comandos de SQL

def query(sql, params=()):
    """Ejecuta un SELECT y devuelve un DataFrame para verlo bonito."""
    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):
    """Ejecuta INSERT/UPDATE/DELETE/DDL y hace commit."""
    cur.execute(sql, params)
    conn.commit()
    print("OK ->", sql.strip().splitlines()[0], "...")
```

Figura 1: Preparación de la Base de Datos

CREATE TABLE comidas

Creamos una tabla llamada **comidas** destinada a almacenar la información de los platillos disponibles en el sistema así mismo se definieron sus campos como lo son: **id_platillo**, que funciona como clave primaria para identificar de manera única cada registro; **nombre**, que almacena el nombre del platillo y se establece como obligatorio mediante la restricción NOT NULL; **categoría**, destinada a clasificar los alimentos; **precio**, que registra el costo del producto con dos decimales de precisión; y **disponible**, un campo booleano que indica si el platillo se encuentra disponible para su venta, tal y como se muestra en la siguiente imagen.



```
17] Creamos la tabla llamada comidas
0 s cur.executescript('''
DROP TABLE IF EXISTS comidas;

CREATE TABLE comidas (
    id_Platillo INTEGER PRIMARY KEY,
    nombre varchar(50) NOT NULL,
    categoria varchar(30),
    precio decimal(6,2),
    disponible boolean
);
''')
conn.commit()
print("Tabla creadas")

... Tabla creadas
```

Figura 2: CREATE TABLE comidas

INSERT INTO Refresco y Comida

Se realizó la inserción de datos en la tabla **comidas** mediante la instrucción INSERT INTO. se agregó un registro correspondiente a un refresco, especificando su nombre, categoría, precio y disponibilidad. Posteriormente, se insertó un segundo registro correspondiente a una sopa, indicando su nombre, categoría y precio, y finalmente se muestra la tabla tal y como se muestra en la siguiente imagen.

```
Realizamos dos insert uno de Refresco y sopa

#insert:de refresco
run("""
INSERT INTO comidas (nombre, categoria, precio, disponible)
VALUES ('Refresco', 'Bebida', 25.00, 1)
""")

conn.commit()

... OK -> INSERT INTO comidas (nombre, categoria, precio, disponible) .

#insert:de comida

run(""" INSERT INTO comidas ( nombre, categoria, precio ) VALUES
('Sopa', 'entrada', 100 );""")

query("select * from comidas")

OK -> INSERT INTO comidas ( nombre, categoria, precio ) VALUES ...

  id_Platillo  nombre  categoria  precio  disponible
0            1  Refresco    Bebida    25         1.0
1            2    Sopa     entrada    100        NaN
```

Figura 3: INSERT INTO Refresco y Comida

UPDATE comidas

Se realizó la actualización de datos sobre la tabla **comidas** utilizando la sentencia **UPDATE**. El objetivo fue modificar el precio del platillo "**Sopa**", cambiando su valor de **\$100.00** a **\$120.00**.

Para llevar a cabo esta modificación se utilizó la cláusula **SET**, la cual permite indicar qué campo será actualizado y el nuevo valor que se asignará. En este caso, se modificó el campo **precio** estableciendo el valor **120**. Además, se empleó la cláusula **WHERE**, cuya función es especificar qué registro debe ser actualizado dentro de la tabla, como se muestra en la siguiente imagen

```
Realizamos la actualizacion del precio de la Sopa con un UPDATE

#update
cur.execute("""
UPDATE comidas
SET precio = 120
WHERE nombre = 'Sopa'
""")

conn.commit()

print("Precio actualizado")
query("SELECT * FROM comidas")

... Precio actualizado
  id_Platillo  nombre  categoria  precio  disponible
0            1  Refresco    Bebida    25         1.0
1            2    Sopa     entrada   120        NaN
```

Figura 4: UPDATE precio

DELETE comidas

Se realizó la eliminación de un registro de la tabla **comidas**, específicamente al **Refresco**, identificado con el **ID 1**. Para llevar a cabo esta acción se utilizó la sentencia **DELETE**, la cual permite eliminar registros que ya no son necesarios, En la instrucción empleada se utilizó la cláusula **WHERE**, cuya función es indicar qué registro debe ser eliminado. En este caso, la condición establecida fue **id_Platillo = 1**, lo que permitió seleccionar únicamente el registro del Refresco. El uso de **WHERE** es muy importante, ya que evita que se eliminen todos los datos de la tabla por error

```
Eliminamos un platillo con el id 1, correspondiente al Refresco y mostramos la tabla nuevamente

#DELETE
cur.execute("""
DELETE FROM comidas
WHERE id_Platillo = 1
""")

conn.commit()

query("SELECT * FROM comidas")

...
  id_Platillo  nombre  categoria  precio  disponible
0            2    Sopa     entrada   120        None
```

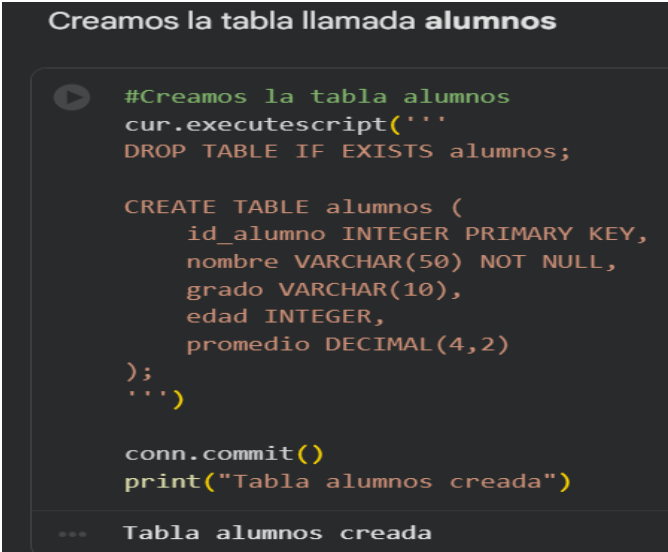
Figura 5: DELETE de Refresco

Escuela

Link de la plataforma Colab: [Escuela.ipynb - Colab](#)

CREATE TABLE alumnos

Se creó la tabla **alumnos**, diseñada para almacenar la información básica de los estudiantes dentro de la base de datos. Para ello, se definieron varios campos que permiten registrar datos importantes de cada alumno. El campo **id_alumno** funciona como identificador único para cada registro, mientras que **nombre** almacena el nombre del estudiante y es un dato obligatorio. Además, se incluye el campo **grado** para indicar el nivel o grupo al que pertenece el alumno, **edad** para registrar los años cumplidos y **promedio** para almacenar su promedio, tal y como se muestra en la siguiente imagen



```

#Creamos la tabla alumnos
cur.executescript('''
DROP TABLE IF EXISTS alumnos;

CREATE TABLE alumnos (
    id_alumno INTEGER PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    grado VARCHAR(10),
    edad INTEGER,
    promedio DECIMAL(4,2)
);
''')

conn.commit()
print("Tabla alumnos creada")

''' Tabla alumnos creada

```

Figura 6: CREATE TABLE alumnos

INSERT alumnos

se procedió a registrar información de tres estudiantes mediante la inserción de nuevos registros. Se agregaron los datos de **Juan Pérez**, **Ana López** y **Carlos Ruiz**, incluyendo información como su grado escolar, edad y promedio académico, así mismo se realizó una consulta para visualizar el contenido de la tabla y verificar que los datos se almacenaran correctamente.

```
run("""
INSERT INTO alumnos
VALUES (1, 'Juan Perez', '3A', 15, 8.5)
""")

run("""
INSERT INTO alumnos
VALUES (2, 'Ana Lopez', '5B', 17, 9.2)
""")

run("""
INSERT INTO alumnos
VALUES (3, 'Carlos Ruiz', '4A', 16, 8.8)
""")

conn.commit()

query("SELECT * FROM alumnos")
```

```
*** OK -> INSERT INTO alumnos ...
OK -> INSERT INTO alumnos ...
OK -> INSERT INTO alumnos ...
```

	id_alumno	nombre	grado	edad	promedio
0	1	Juan Perez	3A	15	8.5
1	2	Ana Lopez	5B	17	9.2
2	3	Carlos Ruiz	4A	16	8.8

Figura 7: *INSERT INTO alumnos*

UPDATE alumnos

Se realizó una actualización en la información de uno de los alumnos registrados. Específicamente, se modificó el **promedio de Juan Pérez**, pasando de **8.5 a 9.8**. Esta acción se llevó a cabo mediante la sentencia **UPDATE**, la cual permite cambiar datos que se encuentran almacenados en una tabla.

Para la modificación se utilizó la cláusula **SET**, cuya función es indicar qué campo será actualizado y cuál será el nuevo valor que se asignará. En este caso, se estableció que el campo **promedio** tomara el valor de **9.8**. Asimismo, se empleó la cláusula **WHERE**, que permite seleccionar el registro específico que se desea modificar.

Realizamos un **UPDATE** al promedio de Juan Perez de 8.5 a 9.8

```
#UPDATE
cur.execute("""
UPDATE alumnos
SET promedio = 9.8
WHERE nombre = 'Juan Perez'
""")

conn.commit()

query("SELECT * FROM alumnos")
```

	id_alumno	nombre	grado	edad	promedio
0	1	Juan Perez	3A	15	9.8
1	2	Ana Lopez	5B	17	9.2
2	3	Carlos Ruiz	4A	16	8.8

Figura 8: UPDATE promedio

DELETE Carlos Ruiz

Se realizó la eliminación de un registro en la tabla **alumnos**, específicamente del estudiante **Carlos Ruiz**. Para esta acción se utilizó la sentencia **DELETE**, la cual permite eliminar información almacenada en una base de datos.

En la instrucción utilizada se empleó la cláusula **WHERE**, cuya función es indicar qué registro debe eliminarse. En este caso, la condición establecida fue **nombre = 'Carlos Ruiz'**, lo que permitió localizar y eliminar únicamente la información correspondiente a ese alumno. El uso de **WHERE** evita que se eliminen todos los registros de la tabla por error.

Eliminamos de la tabla a Carlos Ruiz y mostramos la tabla

```
#DELETE
cur.execute("""
DELETE FROM alumnos
WHERE nombre = 'Carlos Ruiz'
""")

conn.commit()

query("SELECT * FROM alumnos")
```

	id_alumno	nombre	grado	edad	promedio
0	1	Juan Perez	3A	15	9.8
1	2	Ana Lopez	5B	17	9.2

Figura 9: DELETE Carlos Ruiz

Tienda Escolar

Link de plataforma Colab: [Tienda escolar.ipynb - Colab](#)

CREATE TABLE productos

En este ejercicio al igual que los anteriores se creó la tabla **productos** para almacenar la información de los productos disponibles, para ello se definieron varios campos que permiten registrar y organizar los datos de manera adecuada. El campo **id_producto** se estableció como identificador único de cada producto, mientras que **nombre** almacena el nombre del artículo y es un dato obligatorio. Además, se incluyó el campo **tipo** para clasificar los productos según su categoría, **precio** para registrar su costo y **existencia** para llevar un control de la cantidad disponible en inventario.

```
#Creamos la tabla productos
cur.executescript('''
DROP TABLE IF EXISTS productos;

CREATE TABLE productos (
    id_producto INTEGER PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    tipo VARCHAR(30),
    precio DECIMAL(6,2),
    existencia INTEGER
);
''')

conn.commit()
print("Tabla productos creada")

Tabla productos creada
```

Figura 10: CREATE TABLE productos

INSERT INTO productos

se registraron tres artículos para almacenar información relacionada con el inventario. Los productos agregados fueron **Papas**, **Agua** y **Galletas**, cada uno con sus respectivos datos de categoría, precio y cantidad disponible. Posteriormente, se realizó una consulta para verificar que la información se hubiera guardado correctamente, como se muestra en la siguiente imagen.

```
run("""
INSERT INTO productos
VALUES (1, 'Papas', 'Snack', 18.50, 20)
""")

run("""
INSERT INTO productos
VALUES (2, 'Agua', 'Bebida', 12.00, 30)
""")

run("""
INSERT INTO productos
VALUES (3, 'Galletas', 'Snack', 15.00, 25)
""")

conn.commit()

query("SELECT * FROM productos")
```

OK -> INSERT INTO productos ...
OK -> INSERT INTO productos ...
OK -> INSERT INTO productos ...

	id_producto	nombre	tipo	precio	existencia
0	1	Papas	Snack	18.5	20
1	2	Agua	Bebida	12.0	30
2	3	Galletas	Snack	15.0	25

Figura 11: INSERT INTO productos

UPDATE productos

se realizó una actualización en la tabla **productos** para modificar la cantidad disponible del producto **Papas**. La existencia de este artículo se incrementó de **20 a 50 unidades**, reflejando un aumento en el inventario disponible, para después consultar la tabla y verificar los cambios realizados.

```
#UPDATE
cur.execute("""
UPDATE productos
SET existencia = 50
WHERE nombre = 'Papas'
""")

conn.commit()

query("SELECT * FROM productos")
```

	id_producto	nombre	tipo	precio	existencia
0	1	Papas	Snack	18.5	50
1	2	Agua	Bebida	12.0	30
2	3	Galletas	Snack	15.0	25

Figura 12: UPDATE existencia

DELETE Galletas

Se realizó la eliminación del producto **Galletas** de la tabla **productos** con el objetivo de actualizar la información del inventario. Después de ejecutar la eliminación, se consultó nuevamente la tabla para verificar que el registro hubiera sido eliminado correctamente.

```
#DELETE
cur.execute("""
DELETE FROM productos
WHERE nombre = 'Galletas'
""")

conn.commit()

query("SELECT * FROM productos")
```

	id_producto	nombre	tipo	precio	existencia
0	1	Papas	Snack	18.5	50
1	2	Agua	Bebida	12.0	30

Figura 13: DELETE Galletas

Conclusión

La realización de esta práctica permitió comprender el funcionamiento de las bases de datos utilizando Python, SQLite y la plataforma Google Colab. A través de la creación de tablas y la aplicación de operaciones como inserción, consulta, actualización y eliminación de registros, así mismo permitió reforzar las habilidades en el manejo de Bases de Datos.

Además, el uso de **Google Colab** ayudó a familiarizarse con un entorno de trabajo basado en la nube, facilitando la ejecución de código y la visualización de resultados sin necesidad de realizar instalaciones complejas.